LAPORAN PRAKTIKUM

APLIKASI BERBASIS PLATFORM

Tugas Praktikum 7

GIT Branch & Buat Website

Disusun oleh:

Ghilbran Alfaries Pryma

2311102267

S1 IF-04

Dosen Pengampu:

Cahyo Prihantoro, S.Kom., M.Eng

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

INSTITUT TEKNOLOGI TELKOM PURWOKERTO

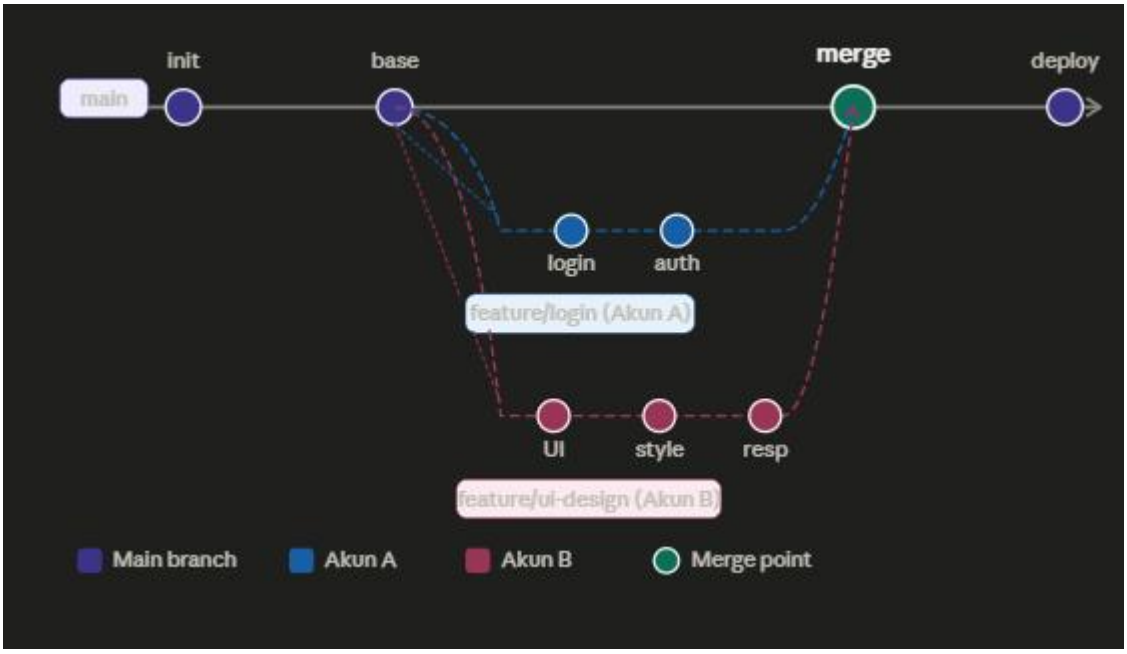
2026/2027

Dasar Teori

1. Git Branch

   Apa itu Git Branch?

   Git Branch adalah fitur di Git yang memungkinkan kamu membuat jalur pengembangan yang terpisah dari kode utama (main/master). Bayangkan seperti cabang pohon — setiap cabang bisa berkembang sendiri tanpa mengganggu cabang lainnya.



   Fungsi dan Keuntungan Git Branch

   Fungsi utama:

   - Memisahkan pengembangan fitur baru dari kode stabil
   - Memungkinkan beberapa developer bekerja secara paralel tanpa konflik
   - Menjadi "zona aman" untuk eksperimen — jika gagal, tinggal hapus branch
   - Memudahkan code review sebelum digabung ke main

   Keuntungan:

   - Kolaborasi tim jadi lebih terstruktur
   - Riwayat perubahan tercatat rapi per fitur
   - Rollback mudah jika ada bug di branch tertentu
   - Mendukung workflow profesional seperti Git Flow atau GitHub Flow

Panduan Lengkap: Git Branch dengan 2 Akun

Setup awal akun A

```
# Clone project dari GitHub
git clone https://github.com/username/nama-project.git
cd nama-project
# Buat dan pindah ke branch baru
git checkout -b feature/login
# Lihat semua branch yang ada
git branch -a
```

Setup awal akun B

```
# Teman clone project yang sama
git clone https://github.com/username/nama-project.git
cd nama-project
# Buat branch untuk fitur berbeda
git checkout -b feature/ui-design
```

Commit dan push masing masing

Akun A

```
git add .
git commit -m "feat: tambah halaman login"
git push origin feature/login
```

Akun B

```
git add .
git commit -m "feat: update tampilan UI homepage"
git push origin feature/ui-design
```

Merge branch ke main (Pull Request)

```
# Pindah ke branch main dulu
git checkout main
git pull origin main
```

```
# Merge branch fitur
git merge feature/login
git merge feature/ui-design

# Jika ada konflik, resolve dulu lalu:
git add .
git commit -m "merge: gabung branch login & ui-design"
git push origin main
```

Input & Output Git Branch

```
Input (perintah)
git branch -b <nama>
git checkout <branch>
git add / commit
git push origin <branch>
git merge <branch>
git pull origin main
git branch -d <nama>
```

```
Output (hasil)
Branch baru terbuat
Berpindah ke branch
Perubahan tersimpan
Branch terupload
Kode tergabung
Kode ter-update
Branch terhapus
```

2. Website dengan Database PostgreSQL (Local)

Berikut adalah contoh website sederhana (sistem manajemen mahasiswa) yang terhubung ke PostgreSQL lokal. Stack yang digunakan: Node.js + Express + PostgreSQL.

Database.js

```
// config/database.js
const { Pool } = require('pg');
require('dotenv').config();

// Konfigurasi koneksi ke PostgreSQL lokal
const pool = new Pool({
    host:     process.env.DB_HOST     || 'localhost',
    port:     parseInt(process.env.DB_PORT) || 5432,
    database: process.env.DB_NAME     || 'db_mahasiswa',
    user:     process.env.DB_USER     || 'postgres',
    password: process.env.DB_PASSWORD || 'password_anda',

    // Pool settings
    max:                    10,
    min:                    2,
    idleTimeoutMillis:      30000,
    connectionTimeoutMillis: 3000,
    allowExitOnIdle:        true,
});

// Test koneksi saat startup
pool.connect((err, client, release) => {
    if (err) {
        console.error('❌ Gagal terhubung ke PostgreSQL:', err.message);
        console.error('   Pastikan PostgreSQL sudah berjalan dan konfigurasi
.env sudah benar');
    } else {
        client.query('SELECT current_database(), version()', (err2, result)
=> {
            release();
            if (!err2) {
                const db = result.rows[0].current_database;
                console.log(`✅ PostgreSQL terhubung — database: "${db}"`);
            }
        });
    }
});

// Log error pool
pool.on('error', (err) => {
    console.error('Unexpected error on idle client', err);
});
```

```
module.exports = pool;
```

Database.sql

```
-- ============================================
-- SETUP DATABASE POSTGRESQL - db_mahasiswa
-- ============================================
-- Jalankan file ini di psql atau pgAdmin:
--   psql -U postgres -f database.sql

-- 1. Buat database (jalankan terpisah sebagai superuser)
-- CREATE DATABASE db_mahasiswa;

-- 2. Sambungkan ke database
\c db_mahasiswa;

-- ============================================
-- TABEL MAHASISWA
-- ============================================
CREATE TABLE IF NOT EXISTS mahasiswa (
    id          SERIAL PRIMARY KEY,
    nim         VARCHAR(12)  UNIQUE NOT NULL,
    nama        VARCHAR(100) NOT NULL,
    jurusan     VARCHAR(50)  NOT NULL
                    CHECK (jurusan IN ('Teknik Informatika', 'Sistem
Informasi', 'Ilmu Komputer')),
    angkatan    INTEGER      NOT NULL CHECK (angkatan BETWEEN 2000 AND
2099),
    ipk         DECIMAL(3,2) NOT NULL DEFAULT 0.00 CHECK (ipk BETWEEN 0 AND
4),
    email       VARCHAR(100),
    no_hp       VARCHAR(20),
    created_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at  TIMESTAMP    NOT NULL DEFAULT CURRENT_TIMESTAMP
);

-- Index untuk mempercepat pencarian
CREATE INDEX IF NOT EXISTS idx_mahasiswa_nim      ON mahasiswa(nim);
CREATE INDEX IF NOT EXISTS idx_mahasiswa_nama     ON mahasiswa USING
gin(to_tsvector('simple', nama));
CREATE INDEX IF NOT EXISTS idx_mahasiswa_jurusan  ON mahasiswa(jurusan);
CREATE INDEX IF NOT EXISTS idx_mahasiswa_angkatan ON mahasiswa(angkatan);
CREATE INDEX IF NOT EXISTS idx_mahasiswa_ipk      ON mahasiswa(ipk DESC);

-- ============================================
-- TABEL MATA KULIAH
-- ============================================
```

```sql
CREATE TABLE IF NOT EXISTS mata_kuliah (
    id          SERIAL PRIMARY KEY,
    kode_mk     VARCHAR(10)  UNIQUE NOT NULL,
    nama_mk     VARCHAR(100) NOT NULL,
    sks         INTEGER      NOT NULL CHECK (sks BETWEEN 1 AND 6),
    semester    INTEGER      NOT NULL CHECK (semester BETWEEN 1 AND 8)
);

-- ==============================================
-- TABEL NILAI (relasi mahasiswa & mata_kuliah)
-- ==============================================
CREATE TABLE IF NOT EXISTS nilai (
    id              SERIAL PRIMARY KEY,
    mahasiswa_id    INTEGER NOT NULL REFERENCES mahasiswa(id)   ON DELETE
CASCADE,
    matakuliah_id   INTEGER NOT NULL REFERENCES mata_kuliah(id) ON DELETE
CASCADE,
    nilai           DECIMAL(5,2) CHECK (nilai BETWEEN 0 AND 100),
    grade           CHAR(2),
    semester        VARCHAR(10) NOT NULL,
    created_at      TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE(mahasiswa_id, matakuliah_id, semester)
);

CREATE INDEX IF NOT EXISTS idx_nilai_mahasiswa ON nilai(mahasiswa_id);

-- ==============================================
-- TRIGGER: otomatis update updated_at
-- ==============================================
CREATE OR REPLACE FUNCTION set_updated_at()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_mahasiswa_updated_at ON mahasiswa;
CREATE TRIGGER trg_mahasiswa_updated_at
    BEFORE UPDATE ON mahasiswa
    FOR EACH ROW EXECUTE FUNCTION set_updated_at();

-- ==============================================
-- DATA CONTOH
-- ==============================================
INSERT INTO mahasiswa (nim, nama, jurusan, angkatan, ipk, email, no_hp)
VALUES
```

```sql
    ('2021001001', 'Budi Santoso',    'Teknik Informatika', 2021, 3.75,
'budi@kampus.ac.id',   '08123456789'),
    ('2021001002', 'Siti Rahayu',     'Sistem Informasi',   2021, 3.50,
'siti@kampus.ac.id',   '08234567890'),
    ('2022001003', 'Ahmad Fauzi',     'Teknik Informatika', 2022, 3.80,
'ahmad@kampus.ac.id',  '08345678901'),
    ('2022001004', 'Dewi Lestari',    'Ilmu Komputer',      2022, 3.20,
'dewi@kampus.ac.id',   '08456789012'),
    ('2023001005', 'Rizky Pratama',   'Teknik Informatika', 2023, 3.65,
'rizky@kampus.ac.id',  '08567890123'),
    ('2023001006', 'Maya Indah',      'Sistem Informasi',   2023, 2.85,
'maya@kampus.ac.id',   '08678901234'),
    ('2024001007', 'Fajar Nugroho',   'Ilmu Komputer',      2024, 3.40,
'fajar@kampus.ac.id',  '08789012345')
ON CONFLICT (nim) DO NOTHING;

INSERT INTO mata_kuliah (kode_mk, nama_mk, sks, semester) VALUES
    ('TI101', 'Algoritma & Pemrograman', 4, 1),
    ('TI102', 'Basis Data',              3, 2),
    ('TI103', 'Jaringan Komputer',       3, 3),
    ('TI104', 'Pemrograman Web',         3, 4),
    ('TI105', 'Kecerdasan Buatan',       3, 5)
ON CONFLICT (kode_mk) DO NOTHING;

-- ==============================================
-- VERIFIKASI
-- ==============================================
SELECT '✅ Database berhasil dibuat!' AS status;
SELECT COUNT(*) AS total_mahasiswa FROM mahasiswa;
SELECT COUNT(*) AS total_mata_kuliah FROM mata_kuliah;
```

Backend REST API (server.js)

```javascript
// server.js - Entry point aplikasi
const express = require('express');
const cors    = require('cors');
const path    = require('path');
require('dotenv').config();

const pool = require('./config/database');

const app  = express();
const PORT = process.env.PORT || 3000;

// =====================
// Middleware
// =====================
app.use(cors());
app.use(express.json());
```

```
app.use(express.urlencoded({ extended: true }));
app.use(express.static(path.join(__dirname, 'public')));

// =====================
// Helper: Validasi Input Mahasiswa
// =====================
function validateMahasiswa(body, isUpdate = false) {
    const errors = [];
    const { nim, nama, jurusan, angkatan, ipk, email, no_hp } = body;

    if (!isUpdate || nim !== undefined) {
        if (!nim) errors.push('NIM wajib diisi');
        else if (!/^\d{10,12}$/.test(nim)) errors.push('NIM harus berupa 10-
12 digit angka');
    }

    if (!isUpdate || nama !== undefined) {
        if (!nama || nama.trim().length < 3) errors.push('Nama wajib diisi
minimal 3 karakter');
    }

    if (!isUpdate || jurusan !== undefined) {
        const jurusanValid = ['Teknik Informatika', 'Sistem Informasi',
'Ilmu Komputer'];
        if (!jurusan || !jurusanValid.includes(jurusan))
errors.push(`Jurusan tidak valid. Pilih: ${jurusanValid.join(', ')}`);
    }

    if (!isUpdate || angkatan !== undefined) {
        const tahun = parseInt(angkatan);
        if (!angkatan || isNaN(tahun) || tahun < 2000 || tahun > new
Date().getFullYear()) {
            errors.push(`Angkatan tidak valid (2000 - ${new
Date().getFullYear()})`);
        }
    }

    if (ipk !== undefined && ipk !== '' && ipk !== null) {
        const ipkNum = parseFloat(ipk);
        if (isNaN(ipkNum) || ipkNum < 0 || ipkNum > 4) errors.push('IPK
harus antara 0.00 - 4.00');
    }

    if (email && email.trim() !== '') {
        if (!/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email)) errors.push('Format
email tidak valid');
    }
```

```
    if (no_hp && no_hp.trim() !== '') {
        if (!/^(08|\+62)\d{8,11}$/.test(no_hp.replace(/[\s-]/g, ''))) {
            errors.push('Format no HP tidak valid (contoh: 08123456789 atau
+6212345678)');
        }
    }

    return errors;
}

// =====================
// HEALTH CHECK
// =====================
app.get('/api/health', async (req, res) => {
    try {
        await pool.query('SELECT 1');
        res.json({
            success: true,
            status: 'OK',
            database: 'connected',
            timestamp: new Date().toISOString()
        });
    } catch (err) {
        res.status(503).json({
            success: false,
            status: 'ERROR',
            database: 'disconnected',
            message: err.message
        });
    }
});

// =====================
// ROUTES - MAHASISWA
// =====================

// GET semua mahasiswa (dengan pencarian & filter)
app.get('/api/mahasiswa', async (req, res) => {
    try {
        const { search, jurusan, angkatan, sort = 'created_at', order =
'DESC' } = req.query;

        const sortAllowed = ['created_at', 'nama', 'nim', 'angkatan',
'ipk'];
        const orderAllowed = ['ASC', 'DESC'];
        const safeSort  = sortAllowed.includes(sort) ? sort : 'created_at';
        const safeOrder = orderAllowed.includes(order.toUpperCase()) ?
order.toUpperCase() : 'DESC';
```

```
        let conditions = [];
        let values = [];
        let idx = 1;

        if (search) {
            conditions.push(`(nama ILIKE $${idx} OR nim ILIKE $${idx})`);
            values.push(`%${search}%`);
            idx++;
        }
        if (jurusan) {
            conditions.push(`jurusan = $${idx}`);
            values.push(jurusan);
            idx++;
        }
        if (angkatan) {
            conditions.push(`angkatan = $${idx}`);
            values.push(parseInt(angkatan));
            idx++;
        }

        const where = conditions.length ? `WHERE ${conditions.join(' AND
')}` : '';
        const query = `SELECT * FROM mahasiswa ${where} ORDER BY ${safeSort}
${safeOrder}`;
        const result = await pool.query(query, values);

        res.json({ success: true, data: result.rows, total: result.rowCount
});
    } catch (err) {
        res.status(500).json({ success: false, message: err.message });
    }
});

// GET mahasiswa berdasarkan ID
app.get('/api/mahasiswa/:id', async (req, res) => {
    try {
        const { id } = req.params;
        if (isNaN(parseInt(id))) {
            return res.status(400).json({ success: false, message: 'ID tidak
valid' });
        }
        const result = await pool.query('SELECT * FROM mahasiswa WHERE id =
$1', [id]);
        if (result.rows.length === 0) {
            return res.status(404).json({ success: false, message:
'Mahasiswa tidak ditemukan' });
        }
```

```
        res.json({ success: true, data: result.rows[0] });
    } catch (err) {
        res.status(500).json({ success: false, message: err.message });
    }
});

// POST - Tambah mahasiswa baru
app.post('/api/mahasiswa', async (req, res) => {
    try {
        const errors = validateMahasiswa(req.body);
        if (errors.length) {
            return res.status(400).json({ success: false, message:
errors.join('; '), errors });
        }

        const { nim, nama, jurusan, angkatan, ipk, email, no_hp } =
req.body;

        const result = await pool.query(
            `INSERT INTO mahasiswa (nim, nama, jurusan, angkatan, ipk,
email, no_hp)
             VALUES ($1, $2, $3, $4, $5, $6, $7)
             RETURNING *`,
            [
                nim.trim(),
                nama.trim(),
                jurusan,
                parseInt(angkatan),
                ipk ? parseFloat(ipk) : 0.00,
                email ? email.trim().toLowerCase() : null,
                no_hp ? no_hp.trim() : null
            ]
        );

        res.status(201).json({
            success: true,
            message: 'Mahasiswa berhasil ditambahkan',
            data: result.rows[0]
        });
    } catch (err) {
        if (err.code === '23505') {
            return res.status(400).json({ success: false, message: 'NIM
sudah terdaftar di sistem' });
        }
        res.status(500).json({ success: false, message: err.message });
    }
});
```

```
// PUT - Update data mahasiswa (full update)
app.put('/api/mahasiswa/:id', async (req, res) => {
    try {
        const { id } = req.params;
        if (isNaN(parseInt(id))) {
            return res.status(400).json({ success: false, message: 'ID tidak
valid' });
        }

        // Cek apakah mahasiswa ada
        const existing = await pool.query('SELECT * FROM mahasiswa WHERE id
= $1', [id]);
        if (existing.rows.length === 0) {
            return res.status(404).json({ success: false, message:
'Mahasiswa tidak ditemukan' });
        }

        const errors = validateMahasiswa(req.body);
        if (errors.length) {
            return res.status(400).json({ success: false, message:
errors.join('; '), errors });
        }

        const { nim, nama, jurusan, angkatan, ipk, email, no_hp } =
req.body;

        const result = await pool.query(
            `UPDATE mahasiswa
             SET nim=$1, nama=$2, jurusan=$3, angkatan=$4, ipk=$5, email=$6,
no_hp=$7, updated_at=NOW()
             WHERE id=$8
             RETURNING *`,
            [
                nim.trim(),
                nama.trim(),
                jurusan,
                parseInt(angkatan),
                ipk !== undefined ? parseFloat(ipk) : existing.rows[0].ipk,
                email ? email.trim().toLowerCase() : null,
                no_hp ? no_hp.trim() : null,
                id
            ]
        );

        res.json({
            success: true,
            message: 'Data mahasiswa berhasil diupdate',
            data: result.rows[0]
```

```
        });
    } catch (err) {
        if (err.code === '23505') {
            return res.status(400).json({ success: false, message: 'NIM
sudah digunakan mahasiswa lain' });
        }
        res.status(500).json({ success: false, message: err.message });
    }
});

// DELETE - Hapus mahasiswa
app.delete('/api/mahasiswa/:id', async (req, res) => {
    try {
        const { id } = req.params;
        if (isNaN(parseInt(id))) {
            return res.status(400).json({ success: false, message: 'ID tidak
valid' });
        }

        const result = await pool.query(
            'DELETE FROM mahasiswa WHERE id=$1 RETURNING id, nim, nama',
[id]
        );

        if (result.rows.length === 0) {
            return res.status(404).json({ success: false, message:
'Mahasiswa tidak ditemukan' });
        }

        res.json({
            success: true,
            message: `Data ${result.rows[0].nama} (${result.rows[0].nim})
berhasil dihapus`
        });
    } catch (err) {
        res.status(500).json({ success: false, message: err.message });
    }
});

// ====================
// ROUTES - STATISTIK
// ====================
app.get('/api/statistik', async (req, res) => {
    try {
        const [statsResult, jurusanResult, angkatanResult] = await
Promise.all([
            pool.query(`
                SELECT
```

```
                    COUNT(*)::int                                AS
total_mahasiswa,
                    COUNT(DISTINCT jurusan)::int                 AS
total_jurusan,
                    ROUND(AVG(ipk)::numeric, 2)                 AS rata_ipk,
                    COUNT(*) FILTER (WHERE ipk >= 3.5)::int     AS
mahasiswa_cumlaude,
                    MAX(ipk)                                     AS
ipk_tertinggi,
                    MIN(NULLIF(ipk, 0))                         AS
ipk_terendah
                FROM mahasiswa
            `),
            pool.query(`
                SELECT jurusan, COUNT(*)::int AS total
                FROM mahasiswa
                GROUP BY jurusan
                ORDER BY total DESC
            `),
            pool.query(`
                SELECT angkatan, COUNT(*)::int AS total
                FROM mahasiswa
                GROUP BY angkatan
                ORDER BY angkatan DESC
            `)
        ]);

        res.json({
            success: true,
            data: {
                ...statsResult.rows[0],
                per_jurusan: jurusanResult.rows,
                per_angkatan: angkatanResult.rows
            }
        });
    } catch (err) {
        res.status(500).json({ success: false, message: err.message });
    }
});

// =====================
// Global Error Handler
// =====================
app.use((err, req, res, next) => {
    console.error('Unhandled error:', err);
    res.status(500).json({ success: false, message: 'Terjadi kesalahan
server internal' });
});
```

```
// 404 handler untuk route yang tidak ada
app.use((req, res) => {
    res.status(404).json({ success: false, message: `Route ${req.method}
${req.path} tidak ditemukan` });
});

// =====================
// Start Server
// =====================
const server = app.listen(PORT, () => {
    console.log(`\n🚀 Server berjalan di http://localhost:${PORT}`);
    console.log(`📋 API Endpoints:`);
    console.log(`   GET    /api/health                - Cek status server &
database`);
    console.log(`   GET    /api/mahasiswa             - Ambil semua
mahasiswa`);
    console.log(`   GET    /api/mahasiswa?search=&jurusan=&angkatan= -
Filter & cari`);
    console.log(`   GET    /api/mahasiswa/:id         - Ambil mahasiswa by
ID`);
    console.log(`   POST   /api/mahasiswa             - Tambah mahasiswa
baru`);
    console.log(`   PUT    /api/mahasiswa/:id         - Update data
mahasiswa`);
    console.log(`   DELETE /api/mahasiswa/:id         - Hapus mahasiswa`);
    console.log(`   GET    /api/statistik             - Lihat statistik
lengkap\n`);
});

// =====================
// Graceful Shutdown
// =====================
const shutdown = async (signal) => {
    console.log(`\n${signal} diterima. Menutup server...`);
    server.close(async () => {
        console.log('HTTP server ditutup.');
        await pool.end();
        console.log('Pool database ditutup. Sampai jumpa!');
        process.exit(0);
    });
};

process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT',  () => shutdown('SIGINT'));
```

Frontend Website (index.html)

```html
<!DOCTYPE html>
<html lang="id">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width,initial-scale=1.0">
<title>Sistem Informasi Mahasiswa</title>
<link
href="https://fonts.googleapis.com/css2?family=Sora:wght@300;400;500;600&fam
ily=Space+Mono:wght@400;700&display=swap" rel="stylesheet">
<style>
  :root {
    --bg: #0e1117;
    --surface: #161b25;
    --card: #1c2333;
    --border: #2a3347;
    --accent: #4f9cf9;
    --accent2: #7c5cbf;
    --green: #3ecf8e;
    --red: #ff4757;
    --amber: #f5a623;
    --text: #e2e8f0;
    --muted: #7a8499;
    --font: 'Sora', sans-serif;
    --mono: 'Space Mono', monospace;
    --radius: 10px;
  }
  * { margin:0; padding:0; box-sizing:border-box; }
  body { background:var(--bg); color:var(--text); font-family:var(--font);
min-height:100vh; }

  /* ===== HEADER ===== */
  header {
    background:var(--surface);
    border-bottom:1px solid var(--border);
    padding:0 2rem;
    display:flex; align-items:center; justify-content:space-between;
    height:60px; position:sticky; top:0; z-index:100;
  }
  .logo { font-family:var(--mono); font-size:13px; color:var(--accent);
letter-spacing:.05em; }
  .logo span { color:var(--muted); }
  .db-status { display:flex; align-items:center; gap:6px; font-size:12px;
color:var(--muted); }
  .dot { width:7px; height:7px; border-radius:50%; background:var(--green);
box-shadow:0 0 6px var(--green); animation:pulse 2s infinite; }
  .dot.err { background:var(--red); box-shadow:0 0 6px var(--red); }
  @keyframes pulse { 0%,100%{opacity:1}50%{opacity:.4} }
```

```
  /* ===== LAYOUT ===== */
  .container { max-width:1140px; margin:0 auto; padding:2rem; }

  /* ===== STATS ===== */
  .stats { display:grid; grid-template-columns:repeat(4,1fr); gap:12px;
margin-bottom:1.75rem; }
  .stat-card {
    background:var(--card); border:1px solid var(--border);
    border-radius:var(--radius); padding:1.2rem;
    transition:border-color .2s, transform .15s;
  }
  .stat-card:hover { border-color:var(--accent); transform:translateY(-1px);
}
  .stat-label { font-size:11px; color:var(--muted); text-
transform:uppercase; letter-spacing:.08em; margin-bottom:6px; }
  .stat-value { font-family:var(--mono); font-size:26px; font-weight:700;
color:var(--accent); }
  .stat-value.green  { color:var(--green); }
  .stat-value.purple { color:#9f7aea; }
  .stat-value.amber  { color:var(--amber); }

  /* ===== PANEL ===== */
  .panel { background:var(--card); border:1px solid var(--border); border-
radius:12px; overflow:hidden; margin-bottom:1.5rem; }
  .panel-header { display:flex; align-items:center; justify-content:space-
between; padding:1rem 1.25rem; border-bottom:1px solid var(--border);
gap:12px; flex-wrap:wrap; }
  .panel-title  { font-size:13px; font-weight:600; letter-spacing:.03em;
white-space:nowrap; }

  /* ===== BUTTON ===== */
  .btn {
    display:inline-flex; align-items:center; gap:5px;
    padding:7px 14px; border-radius:7px;
    font-size:12px; font-weight:500; cursor:pointer;
    border:none; transition:all .15s; font-family:var(--font);
    white-space:nowrap;
  }
  .btn:disabled { opacity:.4; cursor:not-allowed; }
  .btn-primary  { background:var(--accent); color:#fff; }
  .btn-primary:hover:not(:disabled)  { filter:brightness(1.12); }
  .btn-success  { background:var(--green); color:#0e1117; }
  .btn-success:hover:not(:disabled)  { filter:brightness(1.1); }
  .btn-ghost    { background:transparent; color:var(--muted); border:1px
solid var(--border); }
  .btn-ghost:hover:not(:disabled)    { background:var(--surface);
color:var(--text); }
```

```
  .btn-edit     { background:#4f9cf918; color:var(--accent); border:1px
solid #4f9cf940; }
  .btn-edit:hover { background:#4f9cf930; }
  .btn-danger   { background:#ff475718; color:var(--red); border:1px solid
#ff475740; }
  .btn-danger:hover { background:#ff475730; }
  .btn-sm { padding:4px 10px; font-size:11px; }

  /* ===== TOOLBAR ===== */
  .toolbar { display:flex; align-items:center; gap:8px; flex-wrap:wrap; }
  .search-wrap { position:relative; }
  .search-wrap input { padding-left:28px; width:210px; }
  .search-icon { position:absolute; left:9px; top:50%;
transform:translateY(-50%); color:var(--muted); font-size:12px; pointer-
events:none; }
  select.filter { background:var(--surface); border:1px solid var(--border);
border-radius:7px; padding:7px 10px; color:var(--text); font-size:12px;
font-family:var(--font); outline:none; transition:border-color .2s; }
  select.filter:focus { border-color:var(--accent); }
  select.filter option { background:var(--surface); }

  /* ===== TABLE ===== */
  .tbl-wrap { overflow-x:auto; }
  table { width:100%; border-collapse:collapse; font-size:13px; }
  th { text-align:left; padding:.7rem 1.25rem; font-size:11px; color:var(--
muted); text-transform:uppercase; letter-spacing:.07em; border-bottom:1px
solid var(--border); font-weight:500; white-space:nowrap; }
  td { padding:.8rem 1.25rem; border-bottom:1px solid #1e2638; }
  tr:last-child td { border-bottom:none; }
  tr:hover td { background:rgba(79,156,249,.03); }
  .nim  { font-family:var(--mono); font-size:11px; color:var(--muted); }
  .badge { display:inline-block; padding:2px 9px; border-radius:4px; font-
size:11px; font-weight:500; }
  .badge-ti { background:#4f9cf920; color:var(--accent); }
  .badge-si { background:#9f7aea20; color:#9f7aea; }
  .badge-ik { background:#3ecf8e20; color:var(--green); }
  .ipk-high { color:var(--green); font-weight:600; }
  .ipk-mid  { color:var(--amber); font-weight:500; }
  .ipk-low  { color:var(--red); }
  .actions  { display:flex; gap:6px; }
  .empty-row td { text-align:center; color:var(--muted); padding:3rem; }

  /* ===== FORM ===== */
  .form-grid { display:grid; grid-template-columns:1fr 1fr; gap:12px;
padding:1.25rem; }
  .form-group { display:flex; flex-direction:column; gap:5px; }
  .form-group.full { grid-column:1/-1; }
```

```
  label { font-size:11px; color:var(--muted); text-transform:uppercase;
letter-spacing:.06em; }
  input, select.form-select {
    background:var(--surface); border:1px solid var(--border);
    border-radius:7px; padding:8px 10px; color:var(--text);
    font-size:13px; font-family:var(--font); outline:none;
    transition:border-color .2s; width:100%;
  }
  input:focus, select.form-select:focus { border-color:var(--accent); }
  input.err-field, select.form-select.err-field { border-color:var(--red); }
  select.form-select option { background:var(--surface); }
  .form-footer { padding:.75rem 1.25rem; border-top:1px solid var(--border);
display:flex; gap:8px; justify-content:flex-end; }
  .field-err { font-size:11px; color:var(--red); margin-top:2px; }

  /* ===== MODAL ===== */
  .modal-bg {
    position:fixed; inset:0; background:#0008;
    display:none; align-items:center; justify-content:center;
    z-index:200; backdrop-filter:blur(3px);
  }
  .modal-bg.open { display:flex; }
  .modal {
    background:var(--card); border:1px solid var(--border);
    border-radius:14px; width:560px; max-width:95vw;
    max-height:90vh; overflow-y:auto;
    animation:modalIn .2s ease;
  }
  @keyframes modalIn {
from{opacity:0;transform:scale(.95)}to{opacity:1;transform:scale(1)} }
  .modal-header { display:flex; align-items:center; justify-content:space-
between; padding:1rem 1.25rem; border-bottom:1px solid var(--border); }
  .modal-title  { font-size:14px; font-weight:600; }
  .modal-close  { background:none; border:none; color:var(--muted); font-
size:18px; cursor:pointer; padding:2px 6px; border-radius:4px; line-
height:1; }
  .modal-close:hover { background:var(--surface); color:var(--text); }

  /* ===== TOAST ===== */
  #toast {
    position:fixed; bottom:24px; right:24px;
    background:var(--card); border:1px solid var(--border);
    border-radius:9px; padding:10px 16px;
    font-size:13px; opacity:0; transform:translateY(8px);
    transition:all .25s; pointer-events:none; z-index:999;
    max-width:320px; word-break:break-word;
  }
  #toast.show { opacity:1; transform:translateY(0); }
```

```
  #toast.ok  { border-color:var(--green); color:var(--green); }
  #toast.err { border-color:var(--red);   color:var(--red); }

  /* ===== LOADER ===== */
  .loading { display:inline-block; width:14px; height:14px; border:2px solid
var(--border); border-top-color:var(--accent); border-radius:50%;
animation:spin .6s linear infinite; }
  @keyframes spin { to{transform:rotate(360deg)} }

  /* ===== RESPONSIVE ===== */
  @media(max-width:700px) {
    .stats { grid-template-columns:1fr 1fr; }
    .form-grid { grid-template-columns:1fr; }
    .form-group.full { grid-column:1; }
    .container { padding:1rem; }
    header { padding:0 1rem; }
  }
</style>
</head>
<body>

<header>
  <div class="logo"><span>mahasiswa</span> </div>
  <div class="db-status">
    <div class="dot" id="db-dot"></div>
    <span id="db-label">Menghubungkan...</span>
  </div>
</header>

<div class="container">

  <!-- STATISTIK -->
  <div class="stats" id="stats">
    <div class="stat-card"><div class="stat-label">Total Mahasiswa</div><div
class="stat-value" id="s-total">—</div></div>
    <div class="stat-card"><div class="stat-label">Rata-rata IPK</div><div
class="stat-value green" id="s-ipk">—</div></div>
    <div class="stat-card"><div class="stat-label">Jurusan</div><div
class="stat-value purple" id="s-jurusan">—</div></div>
    <div class="stat-card"><div class="stat-label">Cum Laude
(≥3.5)</div><div class="stat-value amber" id="s-cumlaude">—</div></div>
  </div>

  <!-- FORM TAMBAH -->
  <div class="panel">
    <div class="panel-header">
      <span class="panel-title">+ Tambah Mahasiswa Baru</span>
```

```
      <button class="btn btn-ghost btn-sm"
onclick="toggleForm()">Tutup</button>
    </div>
    <div id="form-wrap">
      <div class="form-grid">
        <div class="form-group">
          <label>NIM * <span style="color:var(--muted);font-size:10px">(10-
12 digit)</span></label>
          <input id="f-nim" placeholder="2024001001" maxlength="12"
inputmode="numeric">
          <div class="field-err" id="e-nim"></div>
        </div>
        <div class="form-group">
          <label>Nama Lengkap *</label>
          <input id="f-nama" placeholder="Nama mahasiswa">
          <div class="field-err" id="e-nama"></div>
        </div>
        <div class="form-group">
          <label>Jurusan *</label>
          <select id="f-jurusan" class="form-select">
            <option value="">-- Pilih Jurusan --</option>
            <option>Teknik Informatika</option>
            <option>Sistem Informasi</option>
            <option>Ilmu Komputer</option>
          </select>
          <div class="field-err" id="e-jurusan"></div>
        </div>
        <div class="form-group">
          <label>Angkatan *</label>
          <input id="f-angkatan" type="number" placeholder="2024" min="2000"
max="2099" inputmode="numeric">
          <div class="field-err" id="e-angkatan"></div>
        </div>
        <div class="form-group">
          <label>IPK <span style="color:var(--muted);font-size:10px">(0.00 –
4.00)</span></label>
          <input id="f-ipk" type="number" step="0.01" min="0" max="4"
placeholder="3.75">
          <div class="field-err" id="e-ipk"></div>
        </div>
        <div class="form-group">
          <label>Email</label>
          <input id="f-email" type="email" placeholder="nama@kampus.ac.id">
          <div class="field-err" id="e-email"></div>
        </div>
        <div class="form-group full">
          <label>No. HP</label>
```

```
          <input id="f-hp" placeholder="08123456789" maxlength="15"
inputmode="tel">
          <div class="field-err" id="e-hp"></div>
        </div>
      </div>
      <div class="form-footer">
        <button class="btn btn-ghost" onclick="resetForm()">Reset</button>
        <button class="btn btn-primary" id="btn-simpan"
onclick="simpanMahasiswa()">
          Simpan ke Database
        </button>
      </div>
    </div>
  </div>

  <!-- TABEL DATA -->
  <div class="panel">
    <div class="panel-header">
      <span class="panel-title">Data Mahasiswa</span>
      <div class="toolbar">
        <div class="search-wrap">
          <span class="search-icon">⌕</span>
          <input id="search" placeholder="Cari nama / NIM..."
oninput="onSearch()">
        </div>
        <select class="filter" id="filter-jurusan" onchange="onSearch()">
          <option value="">Semua Jurusan</option>
          <option>Teknik Informatika</option>
          <option>Sistem Informasi</option>
          <option>Ilmu Komputer</option>
        </select>
        <select class="filter" id="filter-angkatan" onchange="onSearch()">
          <option value="">Semua Angkatan</option>
        </select>
        <select class="filter" id="filter-sort" onchange="onSearch()">
          <option value="created_at|DESC">Terbaru</option>
          <option value="nama|ASC">Nama A-Z</option>
          <option value="ipk|DESC">IPK Tertinggi</option>
          <option value="angkatan|DESC">Angkatan Terbaru</option>
        </select>
      </div>
    </div>
    <div class="tbl-wrap">
      <table>
        <thead>
          <tr>
            <th>NIM</th>
            <th>Nama</th>
```

```
            <th>Jurusan</th>
            <th>Angkatan</th>
            <th>IPK</th>
            <th>Email</th>
            <th>Aksi</th>
          </tr>
        </thead>
        <tbody id="tbl-body">
          <tr class="empty-row"><td colspan="7"><span
class="loading"></span> Memuat data dari PostgreSQL...</td></tr>
        </tbody>
      </table>
    </div>
    <div id="tbl-footer" style="padding:.75rem 1.25rem; font-size:12px;
color:var(--muted); border-top:1px solid var(--border);
display:none;"></div>
  </div>

</div>

<!-- ===================== MODAL EDIT ===================== -->
<div class="modal-bg" id="modal-edit">
  <div class="modal">
    <div class="modal-header">
      <span class="modal-title">✏ Edit Data Mahasiswa</span>
      <button class="modal-close" onclick="closeModal()">&times;</button>
    </div>
    <div class="form-grid">
      <input type="hidden" id="edit-id">
      <div class="form-group">
        <label>NIM *</label>
        <input id="edit-nim" placeholder="NIM" maxlength="12"
inputmode="numeric">
        <div class="field-err" id="ee-nim"></div>
      </div>
      <div class="form-group">
        <label>Nama Lengkap *</label>
        <input id="edit-nama" placeholder="Nama mahasiswa">
        <div class="field-err" id="ee-nama"></div>
      </div>
      <div class="form-group">
        <label>Jurusan *</label>
        <select id="edit-jurusan" class="form-select">
          <option value="">-- Pilih --</option>
          <option>Teknik Informatika</option>
          <option>Sistem Informasi</option>
          <option>Ilmu Komputer</option>
        </select>
```

```
        <div class="field-err" id="ee-jurusan"></div>
      </div>
      <div class="form-group">
        <label>Angkatan *</label>
        <input id="edit-angkatan" type="number" min="2000" max="2099"
inputmode="numeric">
        <div class="field-err" id="ee-angkatan"></div>
      </div>
      <div class="form-group">
        <label>IPK</label>
        <input id="edit-ipk" type="number" step="0.01" min="0" max="4">
        <div class="field-err" id="ee-ipk"></div>
      </div>
      <div class="form-group">
        <label>Email</label>
        <input id="edit-email" type="email">
        <div class="field-err" id="ee-email"></div>
      </div>
      <div class="form-group full">
        <label>No. HP</label>
        <input id="edit-hp" maxlength="15" inputmode="tel">
        <div class="field-err" id="ee-hp"></div>
      </div>
    </div>
    <div class="form-footer">
      <button class="btn btn-ghost" onclick="closeModal()">Batal</button>
      <button class="btn btn-success" id="btn-update"
onclick="updateMahasiswa()">
        Simpan Perubahan
      </button>
    </div>
  </div>
</div>

<div id="toast"></div>

<script>
const API = '/api';

// ====================
// Init
// ====================
async function init() {
  await checkHealth();
  await loadData();
}

async function checkHealth() {
```

```
  try {
    const r = await fetch(`${API}/health`);
    const j = await r.json();
    const dot = document.getElementById('db-dot');
    const lbl = document.getElementById('db-label');
    if (j.success) {
      dot.className = 'dot';
      lbl.textContent = 'PostgreSQL connected';
    } else {
      dot.className = 'dot err';
      lbl.textContent = 'Database error';
    }
  } catch {
    document.getElementById('db-dot').className = 'dot err';
    document.getElementById('db-label').textContent = 'Server offline';
  }
}

// =====================
// Load & Render Data
// =====================
async function loadData(params = {}) {
  const url = new URL(`${location.origin}${API}/mahasiswa`);
  Object.entries(params).forEach(([k,v]) => v && url.searchParams.set(k,
v));

  try {
    const r    = await fetch(url);
    const json = await r.json();
    const data = json.data || [];
    renderTable(data);
    buildAngkatanFilter(data);
    updateFooter(json.total, Object.values(params).some(Boolean));
    await loadStats();
  } catch(e) {
    showToast('Gagal terhubung ke server', 'err');
    document.getElementById('tbl-body').innerHTML =
      `<tr class="empty-row"><td colspan="7">❌ Gagal memuat data. Periksa
koneksi server.</td></tr>`;
  }
}

function buildAngkatanFilter(data) {
  const sel   = document.getElementById('filter-angkatan');
  const curr  = sel.value;
  const years = [...new Set(data.map(m => m.angkatan))].sort((a,b) => b-a);
  sel.innerHTML = `<option value="">Semua Angkatan</option>` +
```

```
    years.map(y => `<option ${y == curr ? 'selected':''}
value="${y}">${y}</option>`).join('');
}

async function loadStats() {
  try {
    const r    = await fetch(`${API}/statistik`);
    const json = await r.json();
    const d    = json.data;
    document.getElementById('s-total').textContent    = d.total_mahasiswa;
    document.getElementById('s-ipk').textContent      = d.rata_ipk ?? '—';
    document.getElementById('s-jurusan').textContent  = d.total_jurusan;
    document.getElementById('s-cumlaude').textContent =
d.mahasiswa_cumlaude;
  } catch {}
}

function renderTable(data) {
  const tbody = document.getElementById('tbl-body');
  if (!data.length) {
    tbody.innerHTML = `<tr class="empty-row"><td colspan="7">Tidak ada data
mahasiswa ditemukan</td></tr>`;
    return;
  }

  const badgeMap = {
    'Teknik Informatika': 'badge-ti',
    'Sistem Informasi':   'badge-si',
    'Ilmu Komputer':      'badge-ik'
  };

  tbody.innerHTML = data.map(m => {
    const ipkClass = m.ipk >= 3.5 ? 'ipk-high' : m.ipk >= 3.0 ? 'ipk-mid' :
'ipk-low';
    const ipkVal   = m.ipk ? parseFloat(m.ipk).toFixed(2) : '0.00';
    const badge    = badgeMap[m.jurusan] || '';
    return `<tr>
      <td class="nim">${esc(m.nim)}</td>
      <td style="font-weight:500">${esc(m.nama)}</td>
      <td><span class="badge ${badge}">${esc(m.jurusan)}</span></td>
      <td>${m.angkatan}</td>
      <td class="${ipkClass}">${ipkVal}</td>
      <td style="color:var(--muted);font-size:12px">${esc(m.email||'—
')}</td>
      <td>
        <div class="actions">
          <button class="btn btn-edit btn-sm"
onclick="openEdit(${m.id})">Edit</button>
```

```
          <button class="btn btn-danger btn-sm"
onclick="hapus(${m.id},'${esc(m.nama)}')">Hapus</button>
        </div>
      </td>
    </tr>`;
  }).join('');
}

function updateFooter(total, isFiltered) {
  const el = document.getElementById('tbl-footer');
  el.style.display = '';
  el.textContent = isFiltered
    ? `Menampilkan ${total} dari hasil filter`
    : `Total ${total} mahasiswa`;
}

// =====================
// Search & Filter
// =====================
let searchTimer;
function onSearch() {
  clearTimeout(searchTimer);
  searchTimer = setTimeout(() => {
    const [sort, order] = (document.getElementById('filter-sort').value ||
'created_at|DESC').split('|');
    loadData({
      search:   document.getElementById('search').value.trim(),
      jurusan:  document.getElementById('filter-jurusan').value,
      angkatan: document.getElementById('filter-angkatan').value,
      sort, order
    });
  }, 300);
}

// =====================
// TAMBAH MAHASISWA
// =====================
async function simpanMahasiswa() {
  clearFormErrors();
  const body = {
    nim:      document.getElementById('f-nim').value.trim(),
    nama:     document.getElementById('f-nama').value.trim(),
    jurusan:  document.getElementById('f-jurusan').value,
    angkatan: document.getElementById('f-angkatan').value,
    ipk:      document.getElementById('f-ipk').value || 0,
    email:    document.getElementById('f-email').value.trim(),
    no_hp:    document.getElementById('f-hp').value.trim()
  };
```

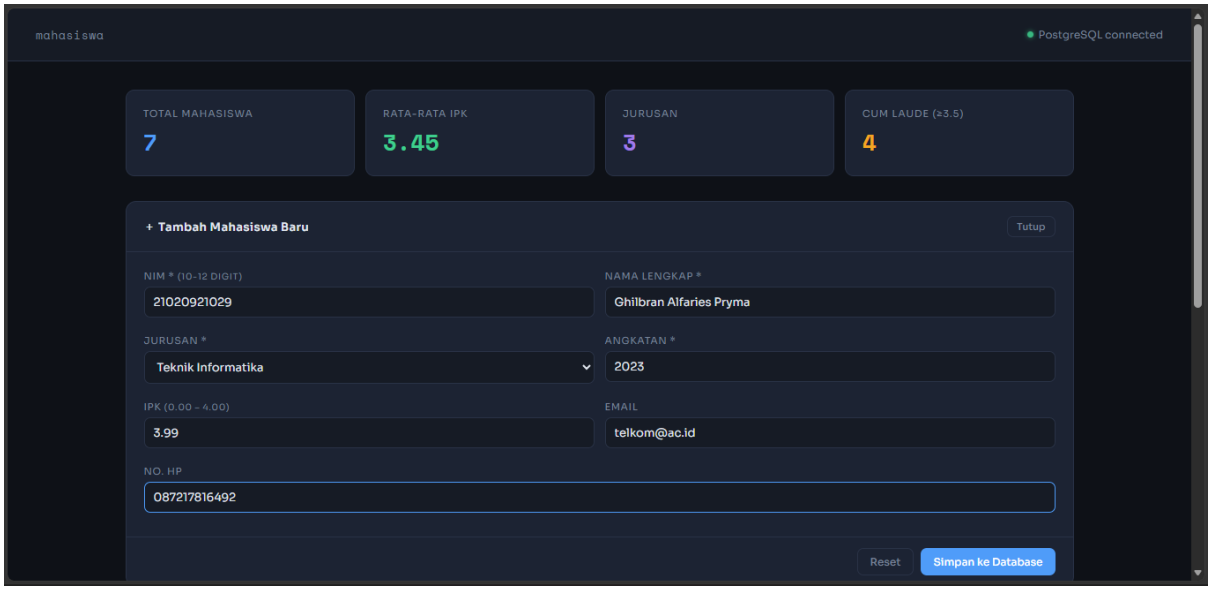
```
  const btn = document.getElementById('btn-simpan');
  btn.disabled = true;
  btn.innerHTML = '<span class="loading"></span> Menyimpan...';

  try {
    const r    = await fetch(`${API}/mahasiswa`, {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
    const json = await r.json();

    if (json.success) {
      showToast('✓ Mahasiswa berhasil disimpan ke database', 'ok');
      resetForm();
      loadData();
    } else {
      showToast(json.message, 'err');
      highlightErrors(json.errors, 'e-');
    }
  } catch(e) {
    showToast('Error: ' + e.message, 'err');
  } finally {
    btn.disabled = false;
    btn.innerHTML = 'Simpan ke Database';
  }
}

// ====================
// EDIT MAHASISWA
// ====================
async function openEdit(id) {
  try {
    const r    = await fetch(`${API}/mahasiswa/${id}`);
    const json = await r.json();
    if (!json.success) { showToast('Gagal memuat data', 'err'); return; }
    const m = json.data;

    document.getElementById('edit-id').value      = m.id;
    document.getElementById('edit-nim').value     = m.nim;
    document.getElementById('edit-nama').value    = m.nama;
    document.getElementById('edit-jurusan').value = m.jurusan;
    document.getElementById('edit-angkatan').value= m.angkatan;
    document.getElementById('edit-ipk').value     = m.ipk;
    document.getElementById('edit-email').value   = m.email || '';
    document.getElementById('edit-hp').value      = m.no_hp || '';
```

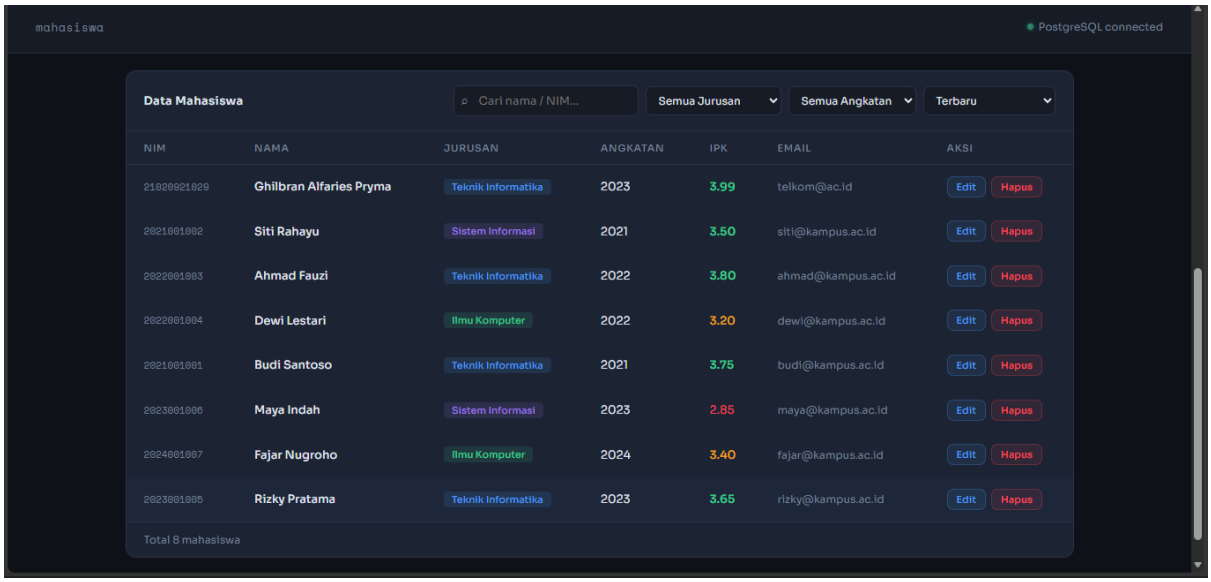
```
    clearEditErrors();
    document.getElementById('modal-edit').classList.add('open');
  } catch(e) {
    showToast('Error: ' + e.message, 'err');
  }
}

function closeModal() {
  document.getElementById('modal-edit').classList.remove('open');
}

async function updateMahasiswa() {
  clearEditErrors();
  const id   = document.getElementById('edit-id').value;
  const body = {
    nim:      document.getElementById('edit-nim').value.trim(),
    nama:     document.getElementById('edit-nama').value.trim(),
    jurusan:  document.getElementById('edit-jurusan').value,
    angkatan: document.getElementById('edit-angkatan').value,
    ipk:      document.getElementById('edit-ipk').value || 0,
    email:    document.getElementById('edit-email').value.trim(),
    no_hp:    document.getElementById('edit-hp').value.trim()
  };

  const btn = document.getElementById('btn-update');
  btn.disabled = true;
  btn.innerHTML = '<span class="loading"></span> Menyimpan...';

  try {
    const r    = await fetch(`${API}/mahasiswa/${id}`, {
      method: 'PUT',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(body)
    });
    const json = await r.json();

    if (json.success) {
      showToast('✓ Data mahasiswa berhasil diupdate', 'ok');
      closeModal();
      loadData();
    } else {
      showToast(json.message, 'err');
      highlightErrors(json.errors, 'ee-');
    }
  } catch(e) {
    showToast('Error: ' + e.message, 'err');
  } finally {
    btn.disabled = false;
```

```
    btn.innerHTML = 'Simpan Perubahan';
  }
}

// =====================
// HAPUS MAHASISWA
// =====================
async function hapus(id, nama) {
  if (!confirm(`Hapus data "${nama}" dari database?\n\nTindakan ini tidak
dapat dibatalkan.`)) return;

  try {
    const r    = await fetch(`${API}/mahasiswa/${id}`, { method: 'DELETE'
});
    const json = await r.json();
    if (json.success) {
      showToast('✓ ' + json.message, 'ok');
      loadData();
    } else {
      showToast(json.message, 'err');
    }
  } catch(e) {
    showToast('Error: ' + e.message, 'err');
  }
}

// =====================
// UI Helpers
// =====================
function resetForm() {
  ['f-nim','f-nama','f-angkatan','f-ipk','f-email','f-hp'].forEach(id => {
    document.getElementById(id).value = '';
    document.getElementById(id).classList.remove('err-field');
  });
  document.getElementById('f-jurusan').value = '';
  clearFormErrors();
}

function toggleForm() {
  const w = document.getElementById('form-wrap');
  w.style.display = w.style.display === 'none' ? '' : 'none';
}

function clearFormErrors() {
  ['nim','nama','jurusan','angkatan','ipk','email','hp'].forEach(f => {
    const el = document.getElementById(`e-${f}`);
    if (el) el.textContent = '';
  });
```

```
}

function clearEditErrors() {
  ['nim','nama','jurusan','angkatan','ipk','email','hp'].forEach(f => {
    const el = document.getElementById(`ee-${f}`);
    if (el) el.textContent = '';
  });
}

function highlightErrors(errors, prefix) {
  if (!errors) return;
  // Simple: just show them as toast if can't map
}

let toastTimer;
function showToast(msg, type = 'ok') {
  const t = document.getElementById('toast');
  t.textContent = msg;
  t.className   = `show ${type}`;
  clearTimeout(toastTimer);
  toastTimer = setTimeout(() => { t.className = ''; }, 3500);
}

// XSS escape
function esc(s) {
  return String(s ?? '')
    .replace(/&/g,'&amp;').replace(/</g,'&lt;').replace(/>/g,'&gt;')
    .replace(/"/g,'&quot;').replace(/'/g,'&#39;');
}

// Close modal on outside click
document.getElementById('modal-edit').addEventListener('click', function(e)
{
  if (e.target === this) closeModal();
});

// Keyboard: Esc to close modal
document.addEventListener('keydown', e => {
  if (e.key === 'Escape') closeModal();
});

// ====================
// Boot
// ====================
init();
</script>
</body>
</html>
```

Screenshoot Program





Deskripsi Program

1. File database.sql (Gudang Data)

File ini berperan sebagai cetak biru (blueprint) dan fondasi utama penyimpanan informasi.

- Fungsi Utama: Mendefinisikan struktur tabel mahasiswa di dalam PostgreSQL.

- Peran Teknis: Mengatur tipe data (seperti VARCHAR untuk nama dan DECIMAL untuk IPK), serta menerapkan aturan keamanan data melalui constraints (seperti CHECK untuk memastikan IPK tidak lebih dari 4.00).
- Otomatisasi: Di dalam file ini, terdapat Trigger dan Function yang berfungsi memperbarui kolom updated_at secara otomatis setiap kali ada perubahan data, sehingga integritas waktu data tetap terjaga.

2. File server.js (Otak / Backend)

File ini bertindak sebagai jembatan atau "polisi lalu lintas" yang mengatur komunikasi antara permintaan pengguna dan pangkalan data.

- Fungsi Utama: Menangani rute API (Application Programming Interface) seperti mengambil data (GET), menambah (POST), mengubah (PUT), dan menghapus (DELETE).
- Validasi: Sebelum data masuk ke database, server.js melakukan pengecekan validasi, misalnya memastikan NIM terdiri dari 10-12 digit dan format email sudah benar.
- Pengolah Logika: File ini juga mengolah data mentah dari database menjadi informasi yang berguna, seperti menghitung statistik rata-rata IPK dan jumlah mahasiswa cum laude melalui endpoint /api/statistik.

3. Koneksi Database (config/database.js / pool)

Bagian ini (yang dipanggil di dalam server.js) berfungsi sebagai pintu akses atau "kabel penghubung" permanen.

- Fungsi Utama: Melakukan inisialisasi koneksi ke PostgreSQL menggunakan kredensial (seperti username dan password) yang disimpan di file .env.
- Manajemen Koneksi: Menggunakan sistem Connection Pool, yang artinya server tidak perlu membuka-tutup koneksi setiap kali ada permintaan, sehingga aplikasi menjadi jauh lebih cepat dan ringan.

4. File index.html

File ini adalah satu-satunya bagian yang berinteraksi langsung dengan pengguna (mahasiswa atau admin).

- Fungsi Utama: Menyediakan antarmuka visual (UI) untuk menampilkan data dalam bentuk tabel dan grafik statistik.

- Logika Frontend (JavaScript): Di dalam file ini terdapat kode JavaScript yang bertugas memanggil API dari server menggunakan fungsi fetch(). Ia juga menangani fitur interaktif seperti pencarian nama secara real-time dan membuka modal untuk mengedit data.
- Estetika (CSS): Mengatur tampilan agar terlihat modern dengan tema dark mode, menggunakan variabel warna agar desain tetap konsisten di seluruh halaman.

Link Github : https://github.com/Ghilbranalf/Praktikum-ABP/tree/e1f5db9a98b9f722df775119f048bfb86e79ae78/ABP%20Praktikum%20Pertemuan%207